Facultad de Defensa Nacional

Licenciatura en CIBERDEFENSA

Lenguajes de Programación

Introducción al lenguaje PHP

v 1.0

Docentes:

Ing. Sebastián Balestrini

Esp. Ing. Gustavo Di Stefano

Carga horaria: 64 hs

Tabla de contenido

Tabla de contenido 1

Introducción 3

¿Que es un servidor web? 3

¿Que es PHP? 3

¿Qué se puede hacer con PHP? 4

Referencia del Lenguaje 6

Sintaxis básica 6

Instrucción de Separación 6

Comentarios 7

Tipos de datos 7

- Booleanos 8
- Enteros 8
- Números Flotantes 8
- Cadenas de caracteres 9
- Arrays 11
- Objetos 12
- NULL 12
- Manipulación de tipos 12
- Forzado de tipos 12

Variables en PHP 13

- Conceptos Básicos 13
- Ambito de las variables 13
- Variables variables 13
- Determinación de los tipos de variables 14

Constantes 14

- Constantes predefinidas 15

Expresiones 16

Operadores 17

- Operadores Aritméticos 17
- Operadores de asignación 18
- Asignación por referencia 18
- Operadores de comparación 18
- Operador de comparación ternario 19
- Operadores de control de errores 19
- Operadores de ejecución 19
- Operadores de incremento/decremento 20
- Operadores lógicos 20
- Operadores para strings 21

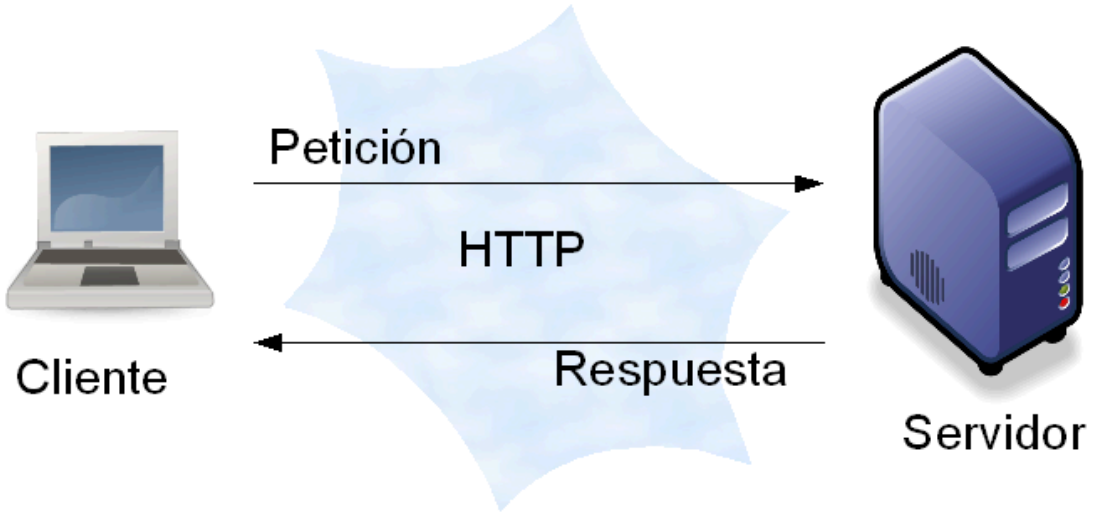
Operadores para arrays 21
Operadores de tipo 22
Operadores bit a bit 22

Estructuras de Control 23
Estructura if() 23
Sentencia else 24
elseif/else if 24
Sentencia while() 25
Sentencia do-while 26
Sentencia for() 26
Sentencia foreach() 27
break 28
continue 29
Sentencia switch 29
return 31

Introducción

¿Que es un servidor web?

Un servidor web es una computadora especialmente configurada para responder a solicitudes de usuarios remotos (clientes) que desean acceder a contenido web. Esta respuesta puede ser una página HTML estática, un archivo, o una respuesta generada dinámicamente mediante lenguajes como PHP.
Ejemplos de servidores web: Apache, Nginx, Microsoft IIS.



¿Que es PHP?

PHP es un acrónimo recursivo que significa **P**HP **H**ipertext **P**reprocesor (Preprocesador de hipertexto). Es un lenguaje de programación del lado del Servidor, gratuito e independiente de plataforma, rápido, con una gran librería de funciones y mucha documentación.
Nota: Una gran parte del contenido de estos apuntes relacionados a PHP están basados en la documentación oficial que se encuentra disponible en http://www.php.net/. Muchas veces se ha optado por transcribir textualmente secciones de la documentación y ejemplos relacionados.
Cuando un navegador solicita una página web en HTML lo que suele suceder a nivel interno es que dicho PC realiza una petición a un servidor Web el cual procesa la petición y devuelve un resultado (la página web solicitada o un error en caso de que no exista o no tenga los permisos necesarios).
Cuando un navegador solicita una página PHP, como PHP es un lenguaje interpretado, el servidor Web necesita de un intérprete para “traducir” lo que el código PHP dice, entonces el servidor Web envía al intérprete PHP la página en PHP y este la traduce a código HTML (entendible por el servidor Web) y el servidor la envía al ordenador solicitante.



Lo que distingue a PHP de lenguajes del lado del "cliente" como JavaScript, es que el código es ejecutado en el servidor, generando salida HTML o TEXTO y enviándolo al cliente. El cliente recibirá los resultados de ejecutar el script, sin ninguna posibilidad de determinar qué código ha producido el resultado recibido. La gran ventaja de este proceso es que el ordenador cliente NUNCA podrá ver el código PHP original realizando una petición, sino que siempre recibirá código HTML brindando seguridad, privacidad y velocidad en la ejecución de determinadas tareas.
Lo mejor de usar PHP es que es extremadamente simple para el principiante, pero a su vez, ofrece muchas características avanzadas para los programadores profesionales. No sientas temor de leer la larga lista de características de PHP que se describen en la próxima sección, en poco tiempo podrás empezar a escribir tus primeros scripts. Pero antes de pasar a ello, veamos un primer ejemplo de código PHP.
El código PHP está entre medio de etiquetas de comienzo y final especiales **<?php** y **?>** que nos permitirán entrar y salir del "modo PHP" a voluntad.

¿Qué se puede hacer con PHP?

PHP puede hacer cualquier cosa que se pueda hacer con un script CGI, como procesar la información de formularios, generar páginas con contenidos dinámicos, o enviar y recibir cookies, manipular bases de datos y mucho más.
Existen principalmente tres campos en los que se usan scripts en PHP:

1. Scripts del lado-servidor. Este es el campo más tradicional y el principal foco de trabajo. Se necesitan tres cosas para que esto funcione. El intérprete PHP (CGI módulo), un servidor web y un navegador. Es necesario hacer funcionar el servidor, con PHP instalado. El resultado del programa PHP se puede obtener a través del navegador, conectándose con el servidor web. La instalación de un servidor web, el intérprete de PHP y normalmente una base de datos como MySQL o PostgreSQL puede realizar instalando y configurando cada uno de

estos componentes o haciendo uso de una WAMP o LAMP, opción altamente recomendada para quienes recién se están iniciando en el desarrollo de aplicaciones Web con PHP. WAMP y LAMP son acrónimos que hacen referencia a Windows, Apache, MySQL, PHP (WAMP) o Linux, Apache, MySQL, PHP (LAMP). Es decir, la W o la L hace referencia al sistema operativo Window o al sistema operativo Linux, la A a Apache que es el nombre del producto comúnmente más utilizado que nos permite transformar nuestra computadora en un servidor web para desarrollo, la M refiere al motor de base de datos MySQL, el más popular (pero no por ello el mejor) motor de base de datos para aplicaciones Web y finalmente la letra P refiere a PHP. La ventaja de instalar una WAMP o LAMP es que generalmente se instala y configura un ambiente de desarrollo de una forma muy simple y rápida. Dependiendo de la WAMP o LAMP elegida (hay muchas) se instalan productos adicionales como puede ser phpMyAdmin, OpenSSL, Mod Perl, etc.

2. Scripts en la línea de comandos. Puede crear un script PHP y correrlo sin necesidad de un servidor web o navegador. Solamente necesita el intérprete PHP para usarlo de esta manera. Este tipo de uso es ideal para scripts ejecutados regularmente desde cron (en *nix o Linux) o el Planificador de tareas (en Windows). Estos scripts también pueden ser usados para tareas simples de procesamiento de texto.
3. Escribir aplicaciones de interfaz gráfica. Probablemente PHP no sea el lenguaje más apropiado para escribir aplicaciones gráficas, pero si conoce bien PHP, y quisiera utilizar algunas características avanzadas en programas clientes, puede utilizar PHP-GTK para escribir dichos programas. También es posible escribir aplicaciones independientes de una plataforma. PHP-GTK es una extensión de PHP, no disponible en la distribución principal.

PHP puede ser utilizado en cualquiera de los principales sistemas operativos del mercado, incluyendo Linux, muchas variantes Unix (incluyendo HP-UX, Solaris y OpenBSD), Microsoft Windows, Mac OS X, RISC OS y probablemente alguno más. PHP soporta la mayoría de servidores web de hoy en día, incluyendo Apache, IIS, y muchos otros. Esto incluye cualquier servidor web que pueda utilizar el binario PHP de FastCGI, como lighttpd y nginx. PHP funciona ya sea como un módulo, o como un procesador de CGI.
De modo que, con PHP tienes la libertad de elegir el sistema operativo y el servidor web de tu gusto. Además, tienes la posibilidad de utilizar programación procedimental, programación orientada a objetos (POO), o una mezcla de ambas.
Una de las características más potentes y destacables de PHP es su soporte para una gran cantidad de bases de datos. Escribir una página web con acceso habilitado a una base de datos es increíblemente simple utilizando una de las extensiones específicas (por ejemplo, para MySQL), o utilizar una capa de abstracción como PDO, o conectarse a cualquier base de datos que soporte el estándar de Conexión Abierta a Bases de Datos por medio de la extensión ODBC. Otras bases de datos podrían utilizar cURL o sockets.
PHP también cuenta con soporte para comunicarse con otros servicios usando protocolos tales como LDAP, IMAP, SNMP, NNTP, POP3, HTTP, COM (en Windows) y muchos otros. También se pueden crear sockets puros e interactuar usando cualquier otro protocolo. PHP soporta WDDX para el intercambio de datos entre lenguajes de programación en web. Y

hablando de interconexión, PHP puede utilizar objetos Java de forma transparente como objetos de PHP.
PHP tiene útiles características de procesamiento de texto, las cuales incluyen las Expresiones Regulares Compatibles de Perl (PCRE), muchas extensiones, y herramientas para el acceso y análisis de documentos XML. PHP estandariza todas las extensiones XML sobre el fundamento sólido de libxml2, y extiende este conjunto de características añadiendo soporte para SimpleXML, XMLReader y XMLWriter.
Como puedes ver, son muchas las características y beneficios de PHP. Hay muchas más que no son mencionadas aquí para no abundar y que podrás encontrar en el manual oficial.

Referencia del Lenguaje

Sintaxis básica

Cuando PHP interpreta un fichero, busca las etiquetas de apertura y fin de script (<?php y ?>), que le indican donde empezar y finalizar la interpretación del código. Este mecanismo permite a PHP ser incrustado en todo tipo de documento. Todo lo que esté fuera de las etiquetas de PHP será ignorado por el intérprete de PHP. Miremos el siguiente ejemplo:

```
1
2
3  <h1>Esto es un bloque HTML</h1>
4
5  <?php
6  echo "Hola munco";
7  ?>
8
9  <p>Esto es un párafo HTML</p>
10
```

Estamos en presencia de un código mixto PHP / HTML. El intérprete PHP va a procesar todo el bloque encerrado entre las etiquetas de apertura y cierre mencionadas **(<?php** y **?>**) . Cuando PHP llega a las etiquetas de cierre ?>, simplemente muestra todo lo que encuentra luego hasta que llega a una nueva etiqueta de apertura.

Instrucción de Separación

PHP requiere que todas las instrucciones se finalicen con el carácter “**;**” (punto y coma). Como habitualmente se estila que las instrucciones ocupen una línea, se suele decir que las líneas en PHP finalizan con punto y coma.

```
13 <?php
14
15     echo "hola mundo";
16     print_r($_REQUEST);
17     $a = $b +5;
18 ?>
```

Comentarios

PHP Soporta comentarios similares al lenguaje “C”, “C++” y estilo Unix/Linux de consola (estilo Perl). Por ejemplo:

```php
<?php

    echo "hola mundo";
    print_r($_REQUEST);
    $a = $b +5;

    // Esta linea es un comentario
    # esto también es otro comentario
?>
```

En resumen, para comentarios de una sola línea se utiliza // o # y para comentarios de múltiples líneas /* para iniciar y */ para finalizar las líneas de comentarios.

```php
<?php
    /* Esto es un comentario
    de múltimples lineas
    */
    echo "Esto es código php";
?>
```

Tipos de datos

PHP soporta ocho tipos primitivos.
Cuatro tipos escalares (simples):

- Boolean
- integer
- float (número de punto flotante, también conocido como double)
- string

Dos tipos compuestos:

- array
- object

Dos tipos especial de PHP lo representan **NULL** y **resource**. El primero de ellos abordaremos más adelante en este apunte.
El manual oficial de PHP introduce también algunos pseudo-tipos por razones de legibilidad: mixed, y number que refieren a argumentos de diferentes tipos o sólo argumentos de tipo numérico (integer o float) respectivamente.
El tipo de una variable usualmente no es declarado por el programador; en cambio, es decidido en tiempo de ejecución por PHP dependiendo del contexto en el que es usado la variable.

Booleanos

Este es el tipo más simple. Un boolean expresa un valor de verdad. Puede ser TRUE or FALSE (verdadero o falso).
Para especificar un literal boolean, use alguna de las palabras clave TRUE o FALSE. Ambas son insensibles a mayúsculas y minúsculas.

Cuando se realizan conversiones a boolean, los siguientes valores son considerados FALSE:

- el boolean FALSE mismo
- el integer 0 (cero)
- el float 0.0 (cero)
- el valor string vacío, y el string "0"
- un array con cero elementos
- un object con cero variables miembro (sólo en PHP 4)
- el tipo especial NULL (incluyendo variables no definidas)
- objetos SimpleXML creados desde etiquetas vacías

Cualquier otro valor es considerado TRUE.

Enteros

Un entero o integer es un número (positivo o negativo) sin decimales. Ejemplo: {..., -2, -1, 0, 1, 2, ...}.
Los integers pueden ser especificados mediante notación decimal (base 10), hexadecimal (base 16), u octal (base 8), opcionalmente precedidos por un signo (- o +).
Para usar la notación octal, se antepone al número un 0 (cero). Para usar la notación hexadecimal, se antepone al número un 0x.
El tamaño de un integer depende de la plataforma, aunque un valor máximo de aproximadamente dos billones es un valor usual (para 32 bits con signo). Las plataformas de 64-bits pueden direccionar valores máximos mucho más elevados.

Conversión a enteros

Para convertir explícitamente un valor a integer, se puede emplear tanto (int) como (integer) en forma de prefijo del valor o variable a convertir. Sin embargo, la mayoría de las veces la conversión no es necesaria, ya que un valor es convertido de forma automática cuando un operador, función o estructura de control necesite un argumento del tipo integer. Un valor también puede ser convertido a integer mediante la función intval().

```
31  <?php
32      $valor = intval($variable);
33      $valor2 = (int)$variable;
34  ?>
```

Números Flotantes

Los números de punto flotante (también conocidos como "flotantes", "dobles" o "números reales") pueden ser especificados usando cualquiera de las siguientes sintaxis:

```
37  <?php
38      $variable_flat = 10.05;
39  ?>
```

El tamaño de un flotante depende de la plataforma, aunque un valor común consiste en un máximo de ~1.8e308 con una precisión de aproximadamente 14 dígitos decimales.

Cadenas de caracteres

Un string (o cadena) es una serie de caracteres, y por lo tanto, un caracter es lo mismo que un byte. Es decir, hay exactamente 256 diferentes caracteres posibles.
PHP no impone ningún límite para el tamaño de un string; el único límite es la memoria disponible en el ordenador donde se está ejecutando PHP.
Un string literal puede ser especificado de cuatro formas diferentes. Veamos las dos más comunes.

Comillas simples

La manera más sencilla de especificar un string es encerrarlo entre comillas simples (el caracter ').
Para especificar una comilla simple literal, se escapa con una barra invertida (\). Para especificar una barra invertida literal, se duplica (\\). Todas las otras instancias de barras invertidas serán tratadas como una barra invertida literal: esto significa que otras secuencias de escape que se puedan utilizar, tales como \r or \n, serán mostradas literalmente como se especifican en lugar de tener cualquier significado especial
A diferencia de las sintaxis de comillas dobles, las variables y las sentencias de escape para caracteres especiales no se expandirán cuando están incluidas dentro de strings con comillas simples.

```
41  <?php
42      $var = 'soy una cadena';
43  ?>
```

Comillas dobles

Si un string está encerrado entre comillas dobles ("), PHP interpretará más sentencias de escape como caracteres especiales:

```
41  <?php
42      $var = "soy una cadena";
43  ?>
```

Sentencia	Significado
\n	Avance de línea (LF o 0x0A (10) en ASCII)
\r	Retorno de carro (CR o 0x0D (13) en ASCII)
\t	Tabulador horizontal (HT o 0x09 (9) en ASCII)

\v	tabulador vertical (VT o 0x0B (11) en ASCII) (desde PHP 5.2.5)
\f	Avance de página (FF o 0x0C (12) en ASCII) (desde PHP 5.2.5)
\\	Barra invertida
\$	Signo del dólar
\"	Comillas dobles

La característica más importante de entrecomillar strings mediante comillas dobles es el hecho que los nombres de las variables son expandidos mostrando su valor en lugar del nombre de la misma. Supongamos que la variable $tampoco = 'ahora si' y $expandiran = 'verán', el resultado de la línea 16 del ejemplo anterior mostraría Las variables ahora si se verán.

```
47  <?php
48      $var = "Bienvenido $nombre !!";
49  ?>
```

Si no requiere la expansión de variables o el uso de sentencias de escape, es más eficiente el uso de cadenas con comillas simples.

Concatenación de Cadenas

Los strings pueden ser concatenados empleando el operador '.' (punto). Fíjese que el operador '+' (suma) no servirá para concatenar.

```
47  <?php
48
49      echo "Hola " . $nombre . " " . $apellido;
50
51  ?>
```

Conversión a string

Un valor puede convertirse a una string mediante el forzado anteponiendo el prefijo (string) o a través de la función strval(). La conversión automática a string tiene lugar en el contexto de una expresión que necesita un string. Esto ocurre cuando se utilizan las funciones echo() o print(), o cuando una variables es comparada con un string.

```
47  <?php
48
49      $valor2 = (string)$variable;
50
51  ?>
```

Acceso a cadenas y modificación por caracteres

Los caracteres dentro de strings puede ser accedidos y modificados especificando la posición de caracter deseado (en base a la posición cero del primer caracter del string) empleando los corchetes de array, como en $str[42]. Piense en este caso que un string se comporta como un array de caracteres. Las funciones substr() y substr_replace() pueden ser empleados para extraer y reemplazar más de un caracter.

Nota: En PHP los arrays o matrices comienzan a contarse desde el elemento 0 (cero). Por lo que el primer carácter de una string $foo será accedido en la forma $foo[0].

Arrays

Un array (matriz) en PHP es en realidad un mapa ordenado. Un mapa es un tipo de datos que asocia valores con claves. Este tipo es optimizado para varios usos diferentes; puede ser usado como una matriz real, una lista (vector), tabla asociativa (caso particular de implementación de un mapa), diccionario, colección, pila, cola y probablemente más. Ya que los valores de un array pueden ser otros arrays, también es posible crear árboles y arrays multidimensionales.
Un valor array puede ser creado por la construcción de lenguaje array(). Éste toma un cierto número de parejas clave => valor separadas con coma.

Una clave puede ser un integer o bien un string. Si una clave es la representación estándar de un integer, será interpretada como tal (es decir, "8" será interpretado como 8, mientras que "08" será interpretado como "08"). Los valores float en key serán truncados a valores tipo integer. Los tipos de arrays indexados y asociativos son el mismo tipo en PHP, el cual pueden contener índices tipo integer o string.
Un valor puede ser de cualquier tipo en PHP.
Si no especifica una clave para un valor dado, entonces es usado el máximo de los índices integer, y la nueva clave será ese valor máximo más 1. Si se especifica una clave que ya tiene un valor asignado, ése valor será sobrescrito.

Creación y modificación con sintaxis de corchetes

Es posible modificar un array existente al definir valores explícitamente en él.
Esto es posible al asignar valores al array al mismo tiempo que se especifica la clave entre corchetes. También es posible omitir la clave, lo que resulta en una pareja de corchetes vacíos ([]).

```php
<?php

    $persona["nombre"] = "Juan perez";

?>
```

```php
<?php

    $dias[0] = "Domingo";
    $dias[1] = "Lunes";
    $dias[2] = "Martes";

?>
```

Arrays son ordenados. El orden puede ser modificado usando varias funciones de ordenado. La función count() puede ser usada para contar el número de elementos en un array.

Objetos

Para crear un nuevo objeto, utilice la declaración new para instanciar una clase:

NULL

El valor especial NULL representa una variable sin valor. NULL es el único posible valor de tipo NULL.
Una variable es considerada null si:

- Se le ha asignado la constante NULL.
- No se ha establecido en un valor todavía.
- Se ha aplicado la función unset().

Manipulación de tipos

PHP no requiere (ni soporta) la definición explicita de tipos en la declaración de variables; el tipo de la variable se determina por el contexto en el cual se emplea la variable. Es decir, si se asigna un valor string a una variable $var, entonces $var se convierte en un string. Si un valor integer es entonces asignado a la misma variable $var, ésta se convierte en integer.
Un ejemplo de la conversión de tipos automática de PHP es el operador suma '+'. Si ambos operandos son float, entonces ambos operandos son evaluados como floats y el resultado será un float. De otra manera, los operandos seran interpretados como integers, y el resultado será entonces integer. Tenga en cuenta que esto no implica que se cambien los tipos de los propios operandos; el único cambio es en como se evalúan los operandos y en el tipo de expresión en sí mismo.

Forzado de tipos

El forzado de tipos en PHP funciona de la misma manera que en C:, donde el nombre del tipo deseado se escribe entre paréntesis antes de la variable que se quiera forzar.
Los siguientes forzados de tipos están permitidos:

- (int), (integer) - forzado a integer
- (bool), (boolean) - forzado a boolean
- (float), (double), (real) - forzado a float
- (string) - forzado a string
- (array) - forzado a array
- (object) - forzado a object
- (unset) - forzado a NULL

Variables en PHP

Conceptos Básicos

En PHP las variables se representan con un signo de dólar seguido por el nombre de la variable. El nombre de la variable es sensible a minúsculas y mayúsculas.
Los nombres de variables siguen las mismas reglas que otras etiquetas en PHP. Un nombre de variable válido tiene que empezar con una letra o un carácter de subrayado (underscore), seguido de cualquier número de letras, números y caracteres de subrayado.
De forma predeterminada, las variables siempre se asignan por valor. Esto significa que cuando se asigna una expresión a una variable, el valor completo de la expresión original se copia en la variable de destino. Esto quiere decir que, por ejemplo, después de asignar el valor de una variable a otra, los cambios que se efectúen a una de esas variables no afectará a la otra.
PHP también ofrece otra forma de asignar valores a las variables: asignar por referencia. Esto significa que la nueva variable simplemente referencia (en otras palabras, "se convierte en un alias de" ó "apunta a") la variable original. Los cambios a la nueva variable afectan a la original, y viceversa.
Para asignar por referencia, simplemente se antepone un signo ampersand (&) al comienzo de la variable cuyo valor se está asignando (la variable fuente). Por ejemplo, el siguiente segmento de código produce la salida 'Mi nombre es Bob' dos veces:

No es necesario inicializar variables en PHP, sin embargo, es una muy buena práctica. Las variables no inicializadas tienen un valor predeterminado de acuerdo a su tipo dependiendo del contexto en el que son usadas - las booleanas se asumen como FALSE, los enteros y flotantes como cero, las cadenas (p.ej. usadas en echo()) se establecen como una cadena vacía y las matrices se convierten en un array vacío.

Ambito de las variables

El ámbito de una variable es el contexto dentro del que la variable está definida. La mayor parte de las variables PHP sólo tienen un ámbito simple. Este ámbito simple también abarca los ficheros incluidos y los requeridos.

Variables variables

A veces es conveniente tener nombres de variables variables. Dicho de otro modo, son nombres de variables que se pueden definir y usar dinámicamente. Una variable normal se establece con una sentencia como:

```
55  <?php
56      $a = 'hola';
57  ?>
```

Una variable variable toma el valor de una variable y lo trata como el nombre de una variable. En el ejemplo anterior, hola, se puede usar como el nombre de una variable utilizando dos signos de dólar. Es decir:

```php
<?php
   $$a = 'mundo';
?>
```

En este momento se han definido y almacenado dos variables en el árbol de símbolos de PHP: $a, que contiene "hola", y $hola, que contiene "mundo". Es más, esta sentencia:

```php
<?php
   echo "$a ${$a}";
?>
```

produce el mismo resultado que:

```php
<?php
   echo "$a $hola";
?>
```

esto quiere decir que ambas producen el resultado: hola mundo.

Determinación de los tipos de variables

Dado que PHP determina los tipos de las variables y los convierte (generalmente) según lo necesita, no siempre resulta obvio de qué tipo es una variable dada en un momento concreto. PHP incluye varias funciones que descubren de qué tipo es una variable: gettype(), is_array(), is_float(), is_int(), is_object(), y is_string().

Constantes

Una constante es un identificador (nombre) para expresar un valor simple. Como el nombre sugiere, este valor no puede variar durante la ejecución del script. (A excepción de las , que en realidad no son constantes). Una constante es sensible a mayúsculas por defecto. Por convención, los identificadores de contantes siempre suelen declararse en mayúsculas.
El nombre de una constante sigue las mismas reglas que cualquier otra etiqueta de PHP. Un nombre de constante válido empieza por una letra o subguión, seguido por cualquier número o letras, números o subguiones.

```php
<?php
    // Defno una constante llamada MI_CONSTANTE
   define("MI_CONSTANTE", "valor");

   // Doy salida al valor de la constante
   echo MI_CONSTANTE;
?>
```

Se puede definir una constante usando la función define() o también declarándola fuera de la clase con const desde PHP. Una vez que la constante está definida, no puede ser cambiada o redefinida en ningún momento.
Solo se puede definir como constantes valores escalares (boolean, integer, float y string).

Para obtener el valor de una constante solo es necesarios especificar su nombre. A diferencia de las variables, no se debe prefijar una constante con el signo $. Puede usar get_defined_constants() para obtener una lista de todas las constantes definidas.
Estas son las diferencias entre constantes y variables:

- Las constantes no llevan el signo dólar ($), como prefijo.
- Las constantes solo pueden ser definidas usando la función define(), y no por simple asignación.
- Las constantes pueden ser definidas y accedidas desde cualquier sitio sin importar las reglas de acceso de variables.
- Las constantes no pueden ser redefinidas o eliminadas una vez se han definido.
- Las constantes solo deberían contener valores escalares.

Constantes predefinidas

PHP ofrece un largo número de constantes predefinidas a cualquier script en ejecución. Muchas de estas constantes, sin embargo, son creadas por diferentes extensiones, y sólo estarán presentes si dichas extensiones están disponibles, bien por carga dinámica o porque han sido compiladas.
Hay siete constantes predefinidas que cambian dependiendo de donde son usadas (no son constantes en el sentido estricto). Por ejemplo el valor de __LINE__ depende en la línea que se use en el script Estas constantes especiales son sensibles a mayúsculas y son las siguientes:

Nombre Descripción

__LINE__ Línea actual en el fichero.

__FILE__ Ruta completa y nombre del fichero. Si se usa dentro de un include, devolverá el nombre del fichero del include. Contiene la ruta absoluta con symlinks (enlaces simbólicos en Linux) resueltos.

__DIR__ Directorio del fichero. Si se utiliza dentro de un include, devolverá el directorio del fichero include.

__FUNCTION__ Nombre de la función. Devuelve el nombre de la función donde fue declarada (sensible a mayúsculas).

__CLASS__ Nombre de la clase. Devuelve el nombre de la clase donde fue declarada (sensible a mayúsculas).

__METHOD__ Nombre del método de la clase. Nombre del método devuelto donde fue declarada. (sensible a mayúsculas).

__NAMESPACE__ Nombre del espacio de nombres actual (sensible a mayúsculas). Esta constante se define en tiempo de compilación (sensible a mayúsculas).

Expresiones

La expresiones son las piedras de construcción más importantes de PHP. En PHP casi todo lo que se escribe es una expresión. La manera más simple y acertada de definir lo que es una expresión es "cualquier cosa que tiene un valor".
Las formas más básicas de expresiones son las constantes y las variables. Cuando se escribe "$a = 5", se está asignando '5' a $a. '5', obviamente, tiene el valor 5, o en otras palabras, '5' es una expresión con el valor de 5 (en este caso, '5' es una constante entera).
Después de esta asignación, se espera que el valor de $a sea 5 también, por lo que si se escribe $b = $a, se espera que esto se comporte tal como si se escribiera $b = 5. En otras palabras, $a es también una expresión con el valor 5. Si todo funciona bien, esto es exactamente lo que sucederá.
PHP lleva las expresiones mucho más allá, de la misma manera que lo hacen otros lenguajes. PHP es un lenguaje orientado a expresiones, en el sentido de que casi todo es una expresión. Considere el ejemplo que ya hemos tratado, '$a = 5'. Es fácil de ver que aquí hay dos valores involucrados, el valor de la constante entera '5', y el valor de $a que ha sido actualizado a 5 también. Aunque la verdad es que existe aquí un valor adicional involucrado, que es el valor de la asignación misma. La asignación evalúa al valor asignado, en este caso 5. En la práctica, esto significa que '$a = 5', sin importar lo que haga, es una expresión con el valor 5. De este modo, escribir algo como '$b = ($a = 5)' es igual que escribir '$a = 5; $b = 5;' (el punto y coma marca el final de una sentencia). Ya que las asignaciones se analizan de derecha a izquierda, también se puede escribir '$b = $a = 5'.
Otro buen ejemplo de orientación a expresiones es el pre- y post-incremento y decremento. Los usuarios de PHP y de otros muchos lenguajes pueden estar familiarizados con la notación variable++ y variable--. Éstos son los operadores de incremento y decremento. En PHP, al igual que en C, hay dos tipos de incrementos - pre-incremento y post-incremento. Ambos esencialmente incrementan la variable, y el efecto sobre la variable es idéntico. La diferencia está con el valor de la expresión de incremento. Pre-incremento, que se escribe '++$variable', evalúa al valor incrementado (PHP incrementa la variable antes de leer su valor, de ahí el nombre de 'pre-incremento'). Post-incremento, que se escribe '$variable++' evalúa el valor original de $variable, antes de que sea incrementado (PHP incrementa la variable después de leer su valor, de ahí el nombre de 'post-incremento').
Un tipo de expresiones muy comunes son las expresiones de comparación. Estas expresiones evalúan si algo es FALSE (falso) o TRUE (verdadero). PHP soporta > (mayor que), >= (mayor o igual que), == (igual), != (distinto), < (menor que) y <= (menor o igual que). El lenguaje también soporta un conjunto de operadores de equivalencia estricta: === (igual y del mismo tipo) y !== (diferente o de distinto tipo). Estas expresiones se usan mayormente dentro de ejecuciones condicionales, tales como la sentencia .
El último ejemplo de expresiones que trataremos aquí es una combinación de expresiones operador-asignación. Ya sabe que si quiere incrementar $a en 1, puede simplemente escribir '$a++' o '++$a'. Pero si lo que quiere es añadirle más de uno, por ejemplo 3, podría escribir '$a++' varias veces, pero esto, obviamente, no es una manera muy eficiente o cómoda. Una práctica mucho más común es escribir '$a = $a + 3'. '$a + 3' evalúa al valor de $a más 3, y se vuelve a asignar a $a, lo que resulta en incrementar $a en 3. En PHP, como en otros lenguajes como C, se puede escribir esto de una manera más abreviada, lo que

con el tiempo se podría convertir en una forma más clara y rápida de entenderlo. Añadir 3 al valor actual de $a puede ser escrito '$a += 3'. Esto significa exactamente "toma el valor de $a, añádele 3 y asígnalo de nuevo a $a". Además de ser más corto y claro, también resulta en una ejecución más rápida. El valor de '$a += 3', al igual que el valor de una asignación normal, es el valor asignado. Observe que NO es 3, sino el valor combinado de $a más 3 (éste es el valor que es asignado a $a). Se puede usar cualquier operador compuesto de dos partes en este modo de operador-asignación, por ejemplo '$a -= 5' (restar 5 del valor de $a), '$b *= 7' (multiplicar el valor de $b por 7), etc.
Algunas expresiones pueden considerarse como sentencias. En este caso, una sentencia tiene la forma 'expr ;', es decir, una expresión seguida de un punto y coma. En '$b = $a = 5;', '$a = 5' es una expresión válida, pero no es una sentencia en sí. Sin embargo, '$b = $a = 5;' es una sentencia válida.
Lo último que vale la pena mencionar es el valor verdadero de las expresiones. En muchos casos, principalmente en ejecuciones condicionales y bucles, no interesa saber el valor específico de la expresión, sino sólo si el valor significa TRUE o FALSE. Las constantes TRUE y FALSE (insensible a mayúsculas-minúsculas) son los dos valores booleanos posibles. Cuando es necesario, una expresión es convertida automáticamente al tipo boolean.

Operadores

Un operador es algo que toma uno más valores (o expresiones, en jerga de programación) y produce otro valor (de modo que la construcción en si misma se convierte en una expresión).
Los operadores se pueden agrupar de acuerdo con el número de valores que toman. Los operadores unarios toman sólo un valor, por ejemplo ! (el operador lógico de negación) o ++ (el operador de incremento). Los operadores binarios toman dos valores, como los familiares operadores aritméticos + (suma) y - (resta), y la mayoría de los operadores de PHP entran en esta categoría. Finalmente, hay sólo un operador ternario, ? :, el cual toma tres valores; usualmente a este se le refiere simplemente como "el operador ternario" (aunque podría tal vez llamarse más correctamente como el operador condicional).vos te d

Operadores Aritméticos

Los operadores aritméticos funcionan tal como los conocemos del uso cotidiano.

Ejemplo	Nombre	Resultado
-$a	Negación	Opuesto de $a.
$a + $b	Adición	Suma de $a y $b.
$a - $b	Sustracción	Diferencia de $a y $b.
$a * $b	Multiplicación	Producto de $a y $b.
$a / $b	División	Cociente de $a y $b.
$a % $b	Módulo	Resto de $a dividido por $b.

El operador de división ("/") devuelve un valor flotante a menos que los dos operandos sean integers (o strings que se conviertan a integers) y los números sean divisibles, en cuyo caso será devuelto un valor integer.

Los operandos del módulo se convierten en integers (por extracción de la parte decimal) antes del procesamiento.

Operadores de asignación

El operador básico de asignación es "=". Se podría inclinar a pensar primero que es como un "igual a". No lo es. Realmente significa que el operando de la izquierda se establece con el valor de la expresión de la derecha (es decir, "se define como").
El valor de una expresión de asignación es el valor asignado. Es decir, el valor de "$a = 3" es de 3. Esto permite hacer algunas cosas intrincadas:

```
86  <?php
87
88      $arreglo[$i+1] = 1975;
89
90      $arrehlo[$i] = $a + $b;
91
92
93  ?>
```

Para arrays, asignar un valor a una clave con nombre se realiza utilizando el operador "=>". La precedencia de este operador es la misma que otros operadores de asignación.
Además del operador básico de asignación, existen "operadores combinados" para todos los de aritmética binaria, unión de arrays y operadores de strings que permiten usar un valor en una expresión y entonces establecer su valor como el resultado de esa expresión.
Tener en cuenta que la asignación copia la variable original en la nueva (asignación por valor), por lo que los cambios en una no afectarán a la otra. Esto también puede tener relevancia si se necesita copiar algo como un gran array dentro de un bucle estrecho.

Asignación por referencia

La asignación por referencia también está soportada, utilizando la sintaxis "$var = &$othervar;". Asignación por referencia significa que ambas variables terminan apuntando a los mismos datos y nada es copiado en ninguna parte.

Operadores de comparación

Los operadores de comparación, como su nombre lo indica, permiten comparar dos valores. La tabla de comparación de tipos disponible en la documentación oficial de PHP muestra ejemplos de varias comparaciones relacionadas con tipos.
Operadores de comparación

Ejemplo	Nombre	Resultado
$a == $b	Igual	TRUE si $a es igual a $b después de la manipulación de tipos.
$a === $b	Idéntico	TRUE si $a es igual a $b, y son del mismo tipo.
$a != $b	Diferente	TRUE si $a no es igual a $b después de la manipulación de tipos.

$a <> $b	Diferente tipos.	TRUE si $a no es igual a $b después de la manipulación de
$a !== $b	No idéntico	TRUE si $a no es igual a $b, o si no son del mismo tipo.
$a < $b	Menor que	TRUE si $a es estrictamente menor que $b.
$a > $b	Mayor que	TRUE si $a es estrictamente mayor que $b.
$a <= $b	Menor o igual que	TRUE si $a es menor o igual que $b.
$a >= $b	Mayor o igual que	TRUE si $a es mayor o igual que $b.

Operador de comparación ternario

Otro operador condicional es el operador "?:" (o ternario).
Ejemplo de asignación de un valor predeterminado

```php
<?php

    // Ejemplo de uso para: Operador Ternario
    $action = (empty($_POST['action'])) ? 'default' : $_POST['action'];

    // Lo anterior es idéntico a esta sentencia if/else
    if (empty($_POST['action'])) {
        $action = 'default';
    } else {
        $action = $_POST['action'];
    }

?>
```

La expresión (expr1) ? (expr2) : (expr3) evalúa a expr2 si expr1 se evalúa como TRUE y a expr3 si expr1 se evalúa como FALSE.
A partir de PHP 5.3, es posible dejar de lado la parte media del operador ternario. La expresión expr1 ?: expr3 retorna expr1 si expr1 se evalúa como TRUE y expr3 si es de otra manera.

Operadores de control de errores

PHP soporta un operador de control de errores: el signo de arroba (@). Cuando se antepone a una expresión en PHP, cualquier mensaje de error que pueden ser generado por esa expresión será ignorado.
Si la propiedad track_errors está activada, cualquier mensaje de error generado por la expresión será guardada en la variable $php_errormsg. Esta variable se sobrescribe en cada error, así que se debe comprobar antes si se desea utilizar.

```php
<?php
    /* Error intencional de archivo */
    $my_file = @file ('non_existent_file') or die ("La apertura de archivo ha fallado: el error fue '$
        php_errormsg'");

    // esto funciona con cualquier expresión, no solo con funciones:
    $value = @$cache[$key]; // no mostrará un error si el índice $key no existe.

?>
```

Operadores de ejecución

PHP soporta un operador de ejecución: las comillas invertidas (``). ¡Note que estas no son las comillas sencillas! PHP intentará ejecutar el contenido entre las comillas invertidas como si se tratara de un comando del shell; la salida será retornada (es decir, no será simplemente volcada como salida; puede ser asignada a una variable). El uso del operador de comillas invertidas es idéntico al de la función shell_exec().

```php
<?php
    $output = `ls -al`; // Ejecuta un comando Linux para mostrar el contenido de un directorio
    echo "<pre>$output</pre>"; // Muestra el resultado de la ejecución del comando (previamente
                               // guardado en $output)
?>
```

Operadores de incremento/decremento

PHP soporta operadores estilo C de pre y post incremento y decremento.
Operadores de incremento/decremento

Ejemplo	Nombre	Efecto
++$a	Pre-incremento	Incrementa $a en uno, y luego retorna $a.
$a++	Post-incremento	Retorna $a, y luego incrementa $a en uno.
--$a	Pre-decremento	Decrementa $a en uno, luego retorna $a.
$a--	Post-decremento	Retorna $a, luego decrementa $a en uno.

Ejemplo de incremento y decremento

```php
<?php
    echo "<h3>Postincremento</h3>";
    $a = 5;
    echo "Debe ser 5: " . $a++ . "<br />\n";
    echo "Debe ser 6: " . $a . "<br />\n";

    echo "<h3>Preincremento</h3>";
    $a = 5;
    echo "Debe ser 6: " . ++$a . "<br />\n";
    echo "Debe ser 6: " . $a . "<br />\n";

    echo "<h3>Postdecremento</h3>";
    $a = 5;
    echo "Debe ser 5: " . $a-- . "<br />\n";
    echo "Debe ser 4: " . $a . "<br />\n";

    echo "<h3>Predecremento</h3>";
    $a = 5;
    echo "Debe ser 4: " . --$a . "<br />\n";
    echo "Debe ser 4: " . $a . "<br />\n";
?>
```

Operadores lógicos

Ejemplo	Nombre	Resultado
$a and $b	And (y)	TRUE si tanto $a como $b son TRUE.

$a or $b	Or (o inclusivo)	TRUE si cualquiera de $a o $b es TRUE.
$a xor $b	Xor (o exclusivo)	TRUE si $a o $b es TRUE, pero no ambos.
!$a	Not (no)	TRUE si $a no es TRUE.
$a && $b	And (y)	TRUE si tanto $a como $b son TRUE.
$a || $b	Or (o inclusivo)	TRUE si cualquiera de $a o $b es TRUE.

La razón para tener las dos variaciones diferentes de los operadores "and" y "or" es que ellos operan con precedencias diferentes.

Operadores para strings

Existen dos operadores para datos tipo string. El primero es el operador de concatenación punto ('.') el cual retorna el resultado de concatenar sus argumentos derecho e izquierdo. El segundo es el operador de asignación sobre concatenación ('.='), el cual añade el argumento del lado derecho al argumento en el lado izquierdo.

```php
<?php
    $a = "Hola ";
    $b = $a . "Mundo!"; // ahora $b contiene " Hola Mundo!"

    $a = " Hola ";
    $a .= "Mundo!";     // ahora $a contiene " Hola Mundo!"
?>

```

Operadores para arrays

Operadores para arrays

Ejemplo	Nombre	Resultado
$a + $b	Unión	Unión de $a y $b.
$a == $b	Igualdad	TRUE si $a y $b tienen las mismas parejas clave/valor.
$a === $b	Identidad	TRUE si $a y $b tienen las mismas parejas clave/valor en el mismo orden y de los mismos tipos.
$a != $b	Desigualdad	TRUE si $a no es igual a $b.
$a <> $b	Desigualdad	TRUE si $a no es igual a $b.
$a !== $b	No-identidad	TRUE si $a no es idéntica a $b.

El operador + devuelve el array del lado derecho añadido al array del lado izquierdo; para las claves que existan en ambos arrays, serán utilizados los elementos del array de la izquierda y serán ignorados los elementos correspondientes del array de la derecha.

```php
<?php
   $a = array("a" => "manzana", "b" => "banana");
   $b = array("a" => "pera", "b" => "naranja", "c" => "cereza");

   $c = $a + $b; // Unión de $a y $b
   echo "Union de \$a y \$b: \n";
   var_dump($c); //var_dump es una funcion de PHP que muestra el contenido de una variable

   $c = $b + $a; // Unión de $b y $a
   echo "Union de \$b y \$a: \n";
   var_dump($c);
?>
```

Cuando sea ejecutado, este script producirá la siguiente salida:

```
Union de $a y $b:
array(3) {
  ["a"]=>
  string(7) "manzana"
  ["b"]=>
  string(6) "banana"
  ["c"]=>
  string(6) "cereza"
}
Union de $b y $a:
array(3) {
  ["a"]=>
  string(4) "pera"
  ["b"]=>
  string(7) "naranja"
  ["c"]=>
  string(6) "cereza"
}
```

Los elementos de los arrays son iguales para la comparación si éstos tienen la misma clave y valor.

Ejemplo de comparación de arrays

```php
<?php
   $a = array("manzana", "banana");
   $b = array(1 => "banana", "0" => "manzana");

   var_dump($a == $b); // bool(true)
   var_dump($a === $b); // bool(false) // No están en el mismo orden
?>
```

Operadores de tipo

instanceof se utiliza para determinar si una variable de PHP es un objeto instanciado de una cierta clase:

Ejemplo de uso de instanceof con clases

```php
<?php
   class MyClass {
   }

   class NotMyClass {
   }

   $a = new MyClass;

   var_dump($a instanceof MyClass);
   var_dump($a instanceof NotMyClass);
?>
```

El resultado del ejemplo sería:
bool(true)
bool(false)

Operadores bit a bit

Los operadores bit a bit permiten la evaluación y la manipulación de bits específicos dentro de un integer. Su uso es muy poco frecuente en el contexto del desarrollo de aplicaciones Web. Igualmente, se documentan en el presente apunte a fin de disponer de una documentación completa de los operadores disponibles en PHP.

Operadores bit a bit

Ejemplo	Nombre	Resultado
$a & $b	And (y)	Los bits que están activos en ambos $a y $b son activados.
$a | $b	Or (o inclusivo)	Los bits que están activos ya sea en $a o en $b son activados.
$a ^ $b	Xor (o exclusivo)	Los bits que están activos en $a o en $b, pero no en ambos, son activados.
~ $a	Not (no)	Los bits que están activos en $a son desactivados, y viceversa. Bits that are set in $a are not set, and vice versa.
$a << $b	Shift left (desplazamiento a izquierda)	Desplaza los bits de $a, $b pasos a la izquierda (cada paso quiere decir "multiplicar por dos").
$a >> $b	Shift right (desplazamiento a derecha)	Desplaza los bits de $a, $b pasos a la derecha (cada paso quiere decir "dividir por dos").

El desplazamiento de bits en PHP es aritmético. Los bits desplazados por fuera de cualquiera de los extremos son descartados. Desplazamientos de izquierda tienen ceros desplazados a la derecha mientras que el bit de signo es desplazado fuera a la izquierda, es decir que no se conserva el signo de un operando. Desplazamientos a la derecha tienen copias del bit de signo desplazado a la izquierda, es decir que se conserva el signo de un operando.

Estructuras de Control

Todo script PHP está construido en base a una serie de sentencias. Una sentencia puede ser una asignación, una llamada de función, un ciclo, una sentencia condicional o incluso una sentencia que no hace nada (una sentencia vacía). Las sentencias generalmente finalizan con un punto-y-coma. Adicionalmente, las sentencias pueden agruparse en un conjunto de sentencias, encapsulándolas entre llaves. Un grupo de sentencias es una sentencia por sí misma también. Los diferentes tipos de sentencias son descritos a continuación.

Estructura if()

El constructor if es una de las características más importantes de muchos lenguajes, incluido PHP. Permite la ejecución condicional de fragmentos de código. PHP dispone de una estructura if que es similar a la de C:

if (expr)
  sentencia

La expresión es evaluada a su valor booleano. Si la expresión se evalúa como TRUE, PHP ejecutará la sentencia y si se evalúa como FALSE la ignorará.

El siguiente ejemplo mostraría a es mayor que b si $a es mayor que $b:

```
200  <?php
201      if ($a > $b)
202          echo "a es mayor que b";
203  ?>
```

A menudo se desea tener más de una sentencia para ser ejecutada condicionalmente. Por supuesto, no hay necesidad de envolver cada sentencia con una cláusula if. En cambio, se pueden agrupar varias sentencias en un grupo de sentencias. Por ejemplo, este código mostraría a es mayor que b si $a es mayor que $b y entonces asignaría el valor de $a a $b:

```
207  <?php
208      if ($a > $b) {
209          echo "a es mayor que b";
210          $b = $a;
211      }
212  ?>
```

Las sentencias if pueden anidarse dentro de otra sentencias if infinitamente, lo cual provee completa flexibilidad para la ejecución condicional de diferentes partes del programa.

Sentencia else

Con frecuencia se desea ejecutar una sentencia si una cierta condición se cumple y una sentencia diferente si la condición no se cumple. Esto es para lo que es else. El else extiende una sentencia if para ejecutar una sentencia en caso que la expresión de la sentencia if se evalúe como FALSE. Por ejemplo, el siguiente código deberá mostrar 'a es mayor que b' si $a es mayor que $b y 'a NO es mayor que b' en el caso contrario:

```php
<?php
   if ($a > $b) {
      echo "a es mayor que b";
   }
   else {
      echo "a NO es mayor que b";
   }
?>
```

La sentencia else sólo es ejecutada si la expresión if es evaluada como FALSE y si hay algunas expresiones elseif - sólo se ejecuta si también todas son evaluadas como FALSE.

elseif/else if

elseif, como su nombre lo sugiere, es una combinación de if y else. Del mismo modo que else, extiende una sentencia if para ejecutar una sentencia diferente en caso que la expresión if original se evalúe como FALSE. Sin embargo, a diferencia de else, esa expresión alternativa sólo se ejecutará si la expresión condicional del elseif se evalúa como TRUE. Por ejemplo, el siguiente código debe mostrar a es mayor que b, a es igual que b o a es menor que b:

```php
<?php
   if ($a > $b) {
      echo "a es mayor que b";
   }
   elseif ($a == $b) {
      echo "a es igual que b";
   }
   else {
      echo "a es menor que b";
   }
?>
```

Puede haber varios elseif dentro de la misma sentencia if. La primera expresión elseif (si hay alguna) que se evalúe como TRUE sería ejecutada.
La sentencia elseif es ejecutada solamente si la expresión if precedente y cualquiera de las expresiones elseif precedentes son evaluadas como FALSE, y la expresión elseif actual se evalúa como TRUE.

Sentencia while()

Los bucles while son el tipo más sencillo de bucle en PHP. La forma básica de una sentencia while es:
while (expr)
   sentencia
El significado de una sentencia while es simple. Le dice a PHP que ejecute las sentencias anidadas, tantas veces como la expresión while se evalúe como TRUE. El valor de la expresión es verificado cada vez al inicio del bucle, por lo que incluso si este valor cambia durante la ejecución de las sentencias anidadas, la ejecución no se detendrá hasta el final de la iteración (cada vez que PHP ejecuta las sentencias contenidas en el bucle es una

iteración). A veces, si la expresión while se evalúa como FALSE desde el principio, las sentencias anidadas no se ejecutarán ni siquiera una vez.
Al igual que con la sentencia if, se pueden agrupar varias instrucciones dentro del mismo bucle while rodeando un grupo de sentencias con llaves.
Mostrando números del 1 al 10 con While

```
237  <?php
238
239      $i = 1;
240      while ($i <= 10) {
241          /* el valor presentado sería $i antes del incremento (post-incremento) */
242          echo $i++ . "\n";
243      }
244
245  ?>
```

Sentencia do-while

Los bucles do-while son muy similares a los bucles while, excepto que la expresión verdadera es verificada al final de cada iteración en lugar que al principio. La diferencia principal con los bucles while es que está garantizado que corra la primera iteración de un bucle do-while (la expresión verdadera sólo es verificada al final de la iteración), mientras que no necesariamente va a correr con un bucle while regular (la expresión verdadera es verificada al principio de cada iteración, si se evalúa como FALSE justo desde el comienzo, la ejecución del bucle terminaría inmediatamente).

Mostrando números del 1 al 10 con do-while

```
<?php
    $i = 1;
    do {
     /* el valor presentado sería $i antes del incremento (post-incremento) */
       echo $i++ . "\n";
    } while ($i <= 10);
?>
```

Sentencia for()

Los bucles for son los ciclos más complejos en PHP. La sintaxis de un bucle for es:

for (expr1; expr2; expr3)
  sentencia

La primera expresión (expr1) es evaluada (ejecutada) una vez incondicionalmente al comienzo del bucle.

En el comienzo de cada iteración, la expr2 es evaluada, Si es TRUE, el blucle continúa y las sentencias anidadas son ejecutadas. Si se evalúa como FALSE, termina la ejecución del bucle.

Al final de cada iteración, la expr3 es evaluada (ejecutada).

Cada una de las expresiones puede esta vacía o contener múltiples expresiones separadas por comas. En la expr2, todas las expresiones separadas por una coma son evaluadas pero el resultado se toma de la última parte. Que la expr2 esté vacía significa que el bucle deberá ser corrido indefinidamente (PHP implícitamente lo considera como TRUE). Esto puede no ser tan inútil como se pudiera pensar, ya que muchas veces se quiere terminar el bucle usando una sentencia condicional en lugar de utilizar la expresión verdadera del for.

Considere los siguientes ejemplos. Todos ellos muestran los números del 1 al 10:

Mostrando números del 1 al 10 con for

```php
<?php
    /* ejemplo 1 */
    for ($i = 1; $i <= 10; $i++) {
        echo $i;
    }

    /* ejemplo 2 */
    for ($i = 1; ; $i++) {
        if ($i > 10) {
            break;
        }
        echo $i;
    }
```

```php
    /* ejemplo 3 */
    $i = 1;
    for (; ; ) {
        if ($i > 10) {
            break;
        }
        echo $i;
        $i++;
    }

    /* ejemplo 4 */
    for ($i = 1, $j = 0; $i <= 10; $j += $i, print $i, $i++);
```

Por supuesto, el primer ejemplo parece ser el mejor (o quizás el segundo), pero se puede encontrar que la posibilidad de usar expresiones vacías en los bucles for resulta útil en muchas ocasiones.

Sentencia foreach()

Allá por la versión 4 de PHP se introdujo un constructor foreach, muy parecido al de Perl y algunos otros lenguajes. Simplemente da un modo fácil de iterar sobre arrays y objetos. foreach trabaja sobre arrays y objetos y resultará en un error al intentar usarlo en una variable con un diferente tipo de datos o una variable no inicializada. Hay dos sintaxis; la segunda es una extensión menor, pero útil de la primera:

Sintaxis 1

```
foreach (array_expression as $value)
    sentencias
```

Sintaxis 2

```
foreach (array_expression as $key => $value)
    sentencias
```

La primera forma recorre el array dado por array_expression. En cada ciclo, el valor del elemento actual es asignado a $value y el puntero interno del array es avanzado en uno (así en el próximo ciclo se estará mirando el siguiente elemento).
La segunda manera hace lo mismo, excepto que la clave del elemento actual será asignada a la variable $key en cada ciclo.
Nota:
A menos que el array este referenciado, foreach opera sobre una copia del array especificado y no sobre el propio array. foreach tiene algunos efectos secundarios sobre el puntero del array. No se puede confiar en el puntero del array durante o después del foreach sin reposicionarlo.
A partir de PHP 5, se pueden modificar fácilmente elementos del array al preceder $value con &. Esto asignará una referencia en lugar de copiar el valor.

```php
<?php

    $arr = array(1, 2, 3, 4);
    foreach ($arr as &$value) {
        $value = $value * 2;
    }

    // $arr ahora es array(2, 4, 6, 8)
    unset($value); // rompe la referencia con el último elemento
?>
```

Atención:
Referencia del $value y el último elemento del array permanecen aún después del bucle foreach. Se recomienda destruirlos con unset().

break

break termina la ejecución de la estructura actual for, foreach, while, do-while o switch.
break acepta un argumento numérico opcional el cual indica de cuantas estructuras anidadas encerradas se debe salir.

```php
    $arr = array('uno', 'dos', 'tres', 'cuatro', 'pare', 'cinco');
    foreach($arr as $val) {
        if ($val == 'pare') {
            break;    /* Se puede también escribir 'break 1;' aquí. */
        }
        echo "$val<br />\n";
    }
```

continue

continue se utiliza dentro de las estructuras de bucle para saltarse el resto de la actual iteración del bucle y continuar la ejecución en la evaluación de la condición y entonces el comienzo de la siguiente iteración.
continue acepta un argumento numérico opcional el cual indica hasta el final de cuantos niveles de bucles cerrados se debe saltar.

```
315    $i = 0;
316    while ($i++ < 5) {
317        echo "Exterior<br />\n";
318        while (1) {
319            echo "Medio<br />\n";
320            while (1) {
321                echo "Interior<br />\n";
322                continue 3;
323            }
324            echo "Esto nunca es mostrado.<br />\n";
325        }
326        echo "Esto tampoco.<br />\n";
327    }
```

Sentencia switch

La sentencia switch es similar a una serie de sentencias IF en la misma expresión. En muchas ocasiones, es posible que se quiera comparar la misma variable (o expresión) con muchos valores diferentes, y ejecutar una parte de código distinta dependiendo de a que valor es igual. Para esto es exactamente la expresión switch.
Ejemplo de estructura if y switch equivalentes

```
347    if ($i == 0) {
348        echo "i es igual a 0";
349    elseif ($i == 1) {
350        echo "i es igual a 1";
351    } elseif ($i == 2) {
352        echo "i es igual a 2";
353    }
354
355    switch ($i) {
356        case 0:
357            echo "i es igual a 0";
358            break;
359        case 1:
360            echo "i es igual a 1";
361            break;
362        case 2:
363            echo "i es igual a 2";
364            break;
365    }
```

Ejemplo de estrutura switch usando strings

```
370    switch ($i) {
371        case "manzana":
372            echo "i es una manzana";
373            break;
374        case "pera":
375            echo "i es una pera";
376            break;
377        case "pastel":
378            echo "i es un pastel";
379            break;
380    }
```

Es importante entender cómo la sentencia switch es ejecutada con el fin de evitar errores. La sentencia switch ejecuta línea por línea (en realidad, sentencia por sentencia). Al principio, ningún código es ejecutado. Sólo cuando una sentencia case es encontrada con un valor que coincide con el valor de la sentencia switch, PHP comienza a ejecutar la sentencias. PHP continúa ejecutando las sentencias hasta el final del bloque switch, o hasta la primera vez que vea una sentencia break. Si no se escribe una sentencia break al final de la lista de sentencias de un caso, PHP seguirá ejecutando las sentencias del caso siguiente. Por ejemplo:

```
384  <?php
385      switch ($i) {
386          case 0:
387              echo "i es igual a 0";
388          case 1:
389              echo "i es igual a 1";
390          case 2:
391              echo "i es igual a 2";
392      }
393  ?>
```

Aquí, si $i es igual a 0, PHP ejecutaría todas las sentencias echo! Si $i es igual a 1, PHP ejecutaría las últimas dos sentencias echo. Se obtendría el comportamiento esperado (se mostraría 'i es igual a 2') sólo si $i es igual a 2. Por lo tanto, es importante no olvidar las sentencias break (aunque es posible que se desee evitar proporcionarlas a propósito bajo determinadas circunstancias).

En una sentencia switch, la condición es evaluada sólo una vez y el resultado es comparado con cada una de las sentencias case. En una sentencia elseif, la condición es evaluada otra vez. Si la condición es más complicada que una simple comparación y/o está en un bucle estrecho, un switch puede ser más rápido.

La lista de sentencias para un caso también puede estar vacía, lo cual simplemente pasa el control a la lista de sentencias para el siguiente caso.

```php
<?php
   switch ($i) {
      case 0:
      case 1:
      case 2:
         echo "i es menor que 3 pero no negativo";
         break;
      case 3:
         echo "i es 3";
   }
?>
```

Un caso especial es el default. Este caso coincide con cualquier cosa que no se haya correspondido por los otros casos. Por ejemplo:

```php
<?php
   switch ($i) {
      case 0:
         echo "i es igual a 0";
         break;
      case 1:
         echo "i es igual a 1";
         break;
      case 2:
         echo "i es igual a 2";
         break;
      default:
         echo "i no es igual a 0, 1 ni 2";
   }
?>
```

La expresión case puede ser cualquier expresión que se evalúa como un tipo simple, es decir, entero o números de punto flotante y strings. Los arrays u objetos no se pueden utilizar aquí a menos que sean desreferenciados a un tipo simple.

return

Si se llama desde una función, la sentencia return() inmediatamente termina la ejecución de la función actual, y retorna su argumento como el valor de la llamada a la función. return() también pondrá fin a la ejecución de una sentencia eval() o a un archivo de script.
Si se llama desde el ámbito global, entonces la ejecución del script actual se termina. Si el archivo script actual fue incluido con include() o require(), entonces el control es pasado de regreso al archivo que hizo el llamado. Además, si el archivo script actual fue incluido, entonces el valor dado a return() será retornado como el valor de la llamada include(). Si return() es llamado desde dentro del archivo script principal, entonces termina la ejecución del script.
Los paréntesis no son obligatorios. Es lo mismo invocar return(); que return;.